

## Summary

This program demonstrates a very small “protocol stack” abstraction built on top of Linux TUN interfaces.

It simulates:

- a protocol instance (proto\_instance)
- an interface abstraction (ifnet)
- packet buffers (mbuf)
- protocol TX/RX processing paths

The code manually builds an IPv4 packet, injects it into a TUN interface, receives it on another TUN interface, and validates the payload.

Conceptually, this resembles a miniature version of how operating systems structure network stacks internally.

---

## High-Level Architecture

The program runs in one of two modes:

- **A (sender)**
  - Creates tun0
  - Builds a custom IPv4 packet
  - Sends it into the TUN device
- **B (receiver)**
  - Creates tun1
  - Waits for packets
  - Filters by source IP
  - Parses payload
  - Verifies "TEST"

---

## Packet Flow

```
INSTANCE_A
↓
build_packet()
↓
proto_output()
↓
tun0
```

↓  
Linux routing/network stack  
↓  
veth routing path  
↓  
tun1  
↓  
rx\_loop()  
↓  
proto\_input()  
↓  
payload verification

---

## Core Design Concepts

This code intentionally mimics kernel networking architecture.

### Real Kernel Concept Simplified Version

|                    |                       |
|--------------------|-----------------------|
| mbuf / sk_buff     | struct mbuf           |
| ifnet / net_device | struct ifnet          |
| protocol instance  | struct proto_instance |
| protocol TX path   | proto_output()        |
| protocol RX path   | proto_input()         |

It is essentially an educational networking-stack simulator.

---

## Detailed Commentary

---

### struct mbuf

```
struct mbuf {  
    int len;  
    unsigned char data[1500];  
};
```

### Purpose

Represents a packet buffer.

### Equivalent In Real Systems

- BSD: mbuf
- Linux: sk\_buff

## Fields

- len
  - packet length
- data
  - raw packet bytes

## Notes

Very simplified:

- fixed-size
  - no chaining
  - no metadata
  - no headroom/tailroom
- 

## struct ifnet

```
struct ifnet {  
    int tun_fd;  
    char name[16];  
};
```

## Purpose

Represents a network interface abstraction.

Stores:

- TUN file descriptor
- interface name

Equivalent to:

- BSD ifnet
  - Linux net\_device
- 

## struct proto\_instance

```
struct proto_instance {  
    struct ifnet ifp;  
    char name[16];  
};
```

## Purpose

Represents a protocol engine instance.

Contains:

- interface
- instance name

This simulates how protocol modules maintain state.

---

## **tun\_alloc()**

int tun\_alloc(char \*dev)

### **Purpose**

Creates a Linux TUN device.

---

### **Key Flags**

IFF\_TUN | IFF\_NO\_PI

### **Meaning**

- IFF\_TUN
    - Layer-3 interface
    - exchanges raw IP packets
  - IFF\_NO\_PI
    - disables Linux packet info header
- 

### **Result**

Read/write operations operate directly on IP packets.

---

## **csum()**

unsigned short csum(void \*buf, int len)

### **Purpose**

Computes IPv4 header checksum.

---

### **How It Works**

IPv4 checksum algorithm:

1. Sum 16-bit words
2. Fold carries

### 3. One's complement result

---

#### Important Detail

This computes only:

- IPv4 header checksum

Not:

- TCP checksum
  - UDP checksum
  - ICMP checksum
- 

#### proto\_output()

```
int proto_output(struct proto_instance *inst,  
                struct mbuf *m)
```

#### Purpose

Transmits a packet through the interface.

---

#### Behavior

##### 1. Log transmission

[INSTANCE\_A] proto\_output: sending X bytes

---

##### 2. Inject packet into TUN

```
write(inst->ifp.tun_fd, m->data, m->len)
```

The kernel treats this as an outbound IP packet.

---

#### Conceptual Meaning

This simulates:

- protocol stack output
  - packet handoff to interface driver
- 

#### proto\_input()

```
void proto_input(struct proto_instance *inst,  
                struct mbuf *m)
```

## Purpose

Processes received packets.

---

## Processing Steps

### 1. Parse IPv4 header

```
struct iphdr *ip = (struct iphdr*)m->data;
```

---

### 2. Print packet metadata

Outputs:

- source IP
  - destination IP
  - packet length
- 

### 3. Extract payload

```
char *payload =  
(char*)m->data + sizeof(struct iphdr);
```

---

### 4. Validate payload

Checks for:

"TEST"

---

## Purpose

Demonstrates:

- protocol parsing
  - payload inspection
  - basic packet validation
- 

## rx\_loop()

```
void rx_loop(struct proto_instance *inst)
```

## Purpose

Receives packets from the TUN interface.

---

## Processing Logic

### 1. Read packet

```
read(inst->ifp.tun_fd, m.data, sizeof(m.data))
```

---

### 2. Store packet length

```
m.len = n;
```

---

### 3. Parse IPv4 header

```
struct iphdr *ip
```

---

### 4. Filter traffic

```
if (ip->saddr != inet_addr("10.0.0.99"))  
    continue;
```

Only processes packets from:

- 10.0.0.99
- 

### 5. Process packet

Calls:

```
proto_input(inst, &m);
```

---

## Important Observation

This loop exits after:

- first matching packet

So it is:

- demonstration code
  - not a persistent service
- 

## build\_packet()

```
void build_packet(struct mbuf *m)
```

## Purpose

Constructs a manual IPv4 packet.

---

## Packet Construction

### 1. Zero packet

```
memset(m, 0, sizeof(*m));
```

---

### 2. Fill IPv4 header

```
ip->version = 4;  
ip->ihl = 5;  
ip->ttl = 64;  
ip->protocol = IPPROTO_ICMP;
```

#### Meaning

- IPv4
  - 20-byte header
  - TTL 64
  - protocol field = ICMP
- 

#### Important Note

The packet is NOT actually valid ICMP.

Only the IP protocol field claims it is ICMP.

No ICMP header exists.

---

### 3. Set addresses

10.0.0.99 -> 10.0.1.99

---

### 4. Add payload

```
memcpy(payload, "TEST", 4);
```

---

### 5. Set total length

```
ip->tot_len =  
    htons(sizeof(struct iphdr) + 4);
```

---



## 6. Compute checksum

```
ip->check = csum(ip, sizeof(struct iphdr));
```

---

### Resulting Packet Layout

```
+-----+
| IPv4 Header |
+-----+
| "TEST"      |
+-----+
```

---

### main()

---

#### Mode A (Sender)

```
./prog A
```

---

#### Setup

Creates:

- tun0

Configures:

- IP address
  - interface state
  - route
- 

#### Sends Packet

```
build_packet(&m);
proto_output(&inst, &m);
```

---

#### Mode B (Receiver)

```
./prog B
```

---

#### Setup

Creates:

- tun1

Configures:

- address
  - route
- 

## **Receives Packet**

Runs:

```
rx_loop(&inst);
```

---

## **Educational Value**

This code is excellent for understanding:

- virtual networking
- TUN interfaces
- packet injection
- IPv4 header structure
- protocol architecture
- kernel-style networking abstractions

It mirrors real kernel networking concepts in a simplified userspace form.

---

## **Major Limitations**

---

### **1. Unsafe Parsing**

Example:

```
struct iphdr *ip = (struct iphdr*)m->data;
```

No validation that:

- packet is large enough
  - header is valid
- 

### **2. Fake ICMP**

Packet claims:

```
IPPROTO_ICMP
```

But contains no ICMP header.

A real network stack may reject this.

---

### 3. No Endianness Validation

Assumes:

- architecture behavior
  - alignment safety
- 

### 4. Uses system()

Example:

```
system("ip link set tun0 up");
```

Problems:

- shell injection risk
- fragile
- slow

Better:

- netlink
  - ioctl APIs
- 

### 5. No Persistent RX Service

Receiver exits after:

- first matching packet
- 

### 6. No Error Recovery

No:

- retries
  - reconnect logic
  - route verification
- 

### 7. Hardcoded Topology

Hardcoded:

- IPs

- interfaces
- routes

Not reusable without edits.

---

## **8. Weak Buffer Design**

mbuf lacks:

- dynamic sizing
  - metadata
  - fragmentation support
  - chaining
- 

## **Suggested Improvements**

---

### **Add Proper ICMP**

Construct:

- ICMP Echo Request
  - ICMP checksum
- 

### **Add Header Validation**

Verify:

- version
  - IHL
  - total length
- 

### **Replace system()**

Use:

- rtnetlink
  - libnl
- 

### **Add Real Event Loop**

Use:

- poll()
- epoll()

---

### Add Protocol Registration

Could evolve into:

IP layer

↓

protocol dispatch

├— ICMP

├— UDP

└— TCP

---

### Add Multi-Interface Support

Would resemble:

- actual router architecture

---

### Overall Assessment

This is a compact educational networking prototype that demonstrates:

- userspace packet handling
- manual IPv4 packet construction
- TUN interfaces
- protocol abstraction layering
- simplified kernel networking architecture

It is especially valuable for learning how:

- packets move through interfaces
- protocol stacks are structured
- virtual interfaces interact with routing

It is not production-ready, but it is an excellent teaching/demo implementation of low-level networking concepts.